

Stage 1 - PFN separation

arch/x86/Kconfig.cpu

変更コード

```
@@ -389,6 +389,7 @@ menuconfig PROCESSOR_SELECT
    config CPU_SUP_INTEL
        default y
        bool "Support Intel processors" if PROCESSOR_SELECT
+       select DYNAMIC_PHYSICAL_MASK if X86_64
        ---help---
            This enables detection, tunings and quirks for Intel
        processors
```

何をしているか

Intel x86-64でDYNAMIC_PHYSICAL_MASKを選択し、物理アドレスのmaskを変更可能なphysical_mask変数から取得させる。

なぜ必要か

検出したKeyID領域をphysical_maskから除外し、KeyID付きPTEから通常のPFNを取り出すため。

arch/x86/include/asm/pgtable_types.h

変更コード

```
@@ -115,13 +115,21 @@
    #define _KERNPG_TABLE_NOENC      (_PAGE_PRESENT | _PAGE_RW
|                                     \
                                     _PAGE_ACCESSED | _PAGE_DIRTY)
```

```

+/*
+ * Keep both the physical address and KeyID when changing
+ * protection, even
+ * though PTE_PFN_MASK excludes KeyID bits when extracting a PFN.
+ * Sign-extend PAGE_MASK for 32-bit PAE physical addresses.
+ */
+#define PTE_PFN_MASK_MAX \
+    (((signed long)PAGE_MASK) & ((1ULL << __PHYSICAL_MASK_SHIFT)
+    - 1))
+
+/*
+ * Set of bits not changed in pte_modify. The pte's
+ * protection key is treated like _PAGE_RW, for
+ * instance, and is not included in this mask since
+ * pte_modify() does modify it.
+ */
-#define _PAGE_CHG_MASK (PTE_PFN_MASK | _PAGE_PCD | _PAGE_PWT
+|
+#define _PAGE_CHG_MASK (PTE_PFN_MASK_MAX | _PAGE_PCD | _PAGE_PWT
+|
+
+    _PAGE_SPECIAL | _PAGE_ACCESSED | _PAGE_DIRTY |
+    _PAGE_SOFT_DIRTY)
#define _HPAGE_CHG_MASK (_PAGE_CHG_MASK | _PAGE_PSE)

```

何をしているか

KeyID領域も含むPTE_PFN_MASK_MAXを追加し、_PAGE_CHG_MASKのアドレス部分に使用する。

なぜ必要か

PFN抽出ではKeyIDを除去する一方、pte_modify()による保護属性変更ではPTEのKeyIDを保持するため。

arch/x86/kernel/cpu/intel.c

変更コード

```
@@ -571,6 +579,21 @@ static void detect_tme(struct cpuinfo_x86 *c)
    mktme_status = MKTME_ENABLED;
}

#ifdef CONFIG_X86_64
+    /*
+     * physical_mask is read-only after init; update it on the
+     boot CPU.
+     */
+    if (c == &boot_cpu_data &&
+        mktme_status == MKTME_ENABLED && nr_keyids) {
+        phys_addr_t keyid_mask;
+
+        /* Use the original CPU width, before subtracting
+        KeyID bits. */
+        keyid_mask = GENMASK_ULL(c->x86_phys_bits - 1,
+                                c->x86_phys_bits -
+                                keyid_bits);
+        physical_mask &= ~keyid_mask;
+    }
+#endif
+
+    /*
+     * KeyID bits effectively lower the number of physical address
+     * bits.  Update cpuinfo_x86::x86_phys_bits accordingly.
```

何をしているか

boot CPUでMKTMEが有効かつKeyIDが存在するとき、元の物理アドレス幅からKeyID maskを計算し、physical_maskから除外する。

なぜ必要か

KeyID 1/2のPTEから同じ通常PFNを取得するため。KeyID位置はx86_phys_bitsを減らす前の幅で計算する。

Stage 2 - userspace MKTME experiment

arch/x86/Kconfig

変更コード

```
@@ -2937,6 +2937,17 @@ config X86_DMA_REMAP
    config HAVE_GENERIC_GUP
        def_bool y

+config X86_MKTME_ALIAS_TEST
+    bool "MKTME KeyID alias experiment device"
+    depends on X86_64 && CPU_SUP_INTEL && EFI && !KASAN
+    default n
+    help
+        Provide /dev/mktme_test for a privileged, single-open Key1/
Key2
+    experiment. Write uses Key1; an ioctl selects Key1 or Key2
for read.
+    Requires exclusive use of KeyIDs 1/2, one package, one
memory node
+    and encryption-capable EFI RAM. Closing the device releases
the page.
+    This is an experimental validation interface, not an MKTME
subsystem.
+
    source "net/Kconfig"

    source "drivers/Kconfig"
```

何をしているか

実験デバイス用の設定項目を追加する。Intel x86-64、EFIあり、KASANなしに限定し、既定では無効にする。

なぜ必要か

専用のKey1/Key2実験コードを選択した構成にだけ組み込むため。

arch/x86/kernel/cpu/intel.c

変更コード

```
@@ -19,6 +19,7 @@
#include <asm/microcode_intel.h>
#include <asm/hwcap2.h>
#include <asm/elf.h>
+include <asm/mktme_test.h>

#ifdef CONFIG_X86_64
#include <linux/topology.h>
@@ -514,6 +515,13 @@ static void detect_vmx_virtcap(struct
cpuinfo_x86 *c)
#define MKTME_UNINITIALIZED          2
static int mktme_status = MKTME_UNINITIALIZED;

+ifdef CONFIG_X86_MKTME_ALIAS_TEST
+bool intel_mktme_test_enabled(void)
+{
+    return READ_ONCE(mktme_status) == MKTME_ENABLED;
+}
+endif
+
static void detect_tme(struct cpuinfo_x86 *c)
{
    u64 tme_activate, tme_policy, tme_crypto_algs;
```

何をしているか

実験用ヘッダーとintel_mktme_test_enabled()を追加し、mktme_statusが有効状態かをREAD_ONCE()で参照する。

なぜ必要か

実験開始時とページアクセス時に、CPU検出処理が判定したMKTMEの状態を確認するため。

arch/x86/mm/Makefile

変更コード

```
@@ -53,3 +53,4 @@ obj-$(CONFIG_PAGE_TABLE_ISOLATION) +=  
pti.o  
obj-$(CONFIG_AMD_MEM_ENCRYPT) += mem_encrypt.o  
obj-$(CONFIG_AMD_MEM_ENCRYPT) += mem_encrypt_identity.o  
obj-$(CONFIG_AMD_MEM_ENCRYPT) += mem_encrypt_boot.o  
+obj-$(CONFIG_X86_MKTME_ALIAS_TEST) += mktme_test.o
```

何をしているか

CONFIG_X86_MKTME_ALIAS_TESTが有効な場合にmktme_test.oを組み込む。

なぜ必要か

設定項目を実験デバイスの実装に接続するため。

arch/x86/platform/efi/efi.c

変更コード

```
@@ -47,6 +47,7 @@  
#include <linux/bcd.h>  
  
#include <asm/setup.h>  
+#include <asm/mktme_test.h>  
#include <asm/efi.h>  
#include <asm/e820/api.h>  
#include <asm/time.h>  
@@ -539,6 +540,7 @@ void __init efi_init(void)  
{  
  
    efi_clean_memmap();  
+    mktme_test_save_efi_map();
```

```
if (efi_enabled(EFI_DBG))
    efi_print_memmap();
```

何をしているか

実験用ヘッダーを読み込み、efi_clean_memmap()直後にmktme_test_save_efi_map()を呼ぶ。

なぜ必要か

通常RAMのEFI記述が失われる前に暗号化可能領域を保存し、実験ページの適否を後から判断するため。

arch/x86/include/asm/mktme_test.h

変更コード

```
/* SPDX-License-Identifier: GPL-2.0 */
#ifndef _ASM_X86_MKTME_TEST_H
#define _ASM_X86_MKTME_TEST_H

#include <linux/types.h>
#include <linux/init.h>

bool intel_mktme_test_enabled(void);

#ifdef CONFIG_X86_MKTME_ALIAS_TEST
void __init mktme_test_save_efi_map(void);
#else
static inline void mktme_test_save_efi_map(void) { }
#endif

#endif
```

何をしているか

MKTME有効状態の参照関数とEFI領域保存関数を宣言する。実験機能が無効な構成では保存関数を空のinline関数にする。

なぜ必要か

CPU検出処理、EFI初期化処理、実験デバイスの間で必要な関数だけを共有するため。

include/uapi/linux/mktme_test.h

変更コード

```
/* SPDX-License-Identifier: GPL-2.0 WITH Linux-syscall-note */
#ifndef _UAPI_LINUX_MKTME_TEST_H
#define _UAPI_LINUX_MKTME_TEST_H

#include <linux/ioctl.h>
#include <linux/types.h>

#define MKTME_TEST_MAX_BYTES 4096

/* Last installed PTE, not a live query; mappings are temporary. */
struct mktme_test_info {
    __u64 pfn;
    __u64 pte;
    __u32 keyid;
    __u32 length;
};

/* write() always uses Key1. read() returns a snapshot from offset
zero. */
#define MKTME_TEST_SET_READ_KEY _IOW('M', 0xe0, __u32)
#define MKTME_TEST_GET_INFO      _IOR('M', 0xe1, struct
mktme_test_info)
/* Release keys/page now, reporting errors; close() also performs
cleanup. */
#define MKTME_TEST_FINISH        _IO('M', 0xe2)

#endif
```

何をしているか

転送上限4096バイト、PFN・PTE・KeyID・長さの情報構造体、読出し鍵選択・情報取得・終了

のioctlを定義する。

なぜ必要か

kernelとuserspaceで同じ操作番号とデータ形式を使い、同じPFNへのKey1/Key2アクセスを操作・確認するため。

arch/x86/mm/mktme_test.c

変更コード

include • 定数 • test_config • test_alias • EFI領域保存先

```
// SPDX-License-Identifier: GPL-2.0
/* Single-open experiment device. No allocator KeyID ownership. */
#define pr_fmt(fmt) "mktme-alias-test: " fmt
#include <linux/capability.h>
#include <linux/compat.h>
#include <linux/cpu.h>
#include <linux/efi.h>
#include <linux/init.h>
#include <linux/miscdevice.h>
#include <linux/mktme_test.h>
#include <linux/mm.h>
#include <linux/module.h>
#include <linux/mutex.h>
#include <linux/slab.h>
#include <linux/suspend.h>
#include <linux/uaccess.h>
#include <linux/vmalloc.h>
#include <linux/workqueue.h>
#include <asm/cacheflush.h>
#include <asm/intel_pconfig.h>
#include <asm/mktme_test.h>
#include <asm/msr.h>
#include <asm/pgtable.h>
#include <asm/tlbflush.h>
```

```
#include <asm/topology.h>

#define TEST_TME_CAPABILITY 0x981
#define TEST_TME_ACTIVATE 0x982
#define TEST_ICX_STATUS 0x6f
#define TEST_EFI_CPU_CRYPT0 BIT_ULL(19)

struct test_config {
    u64 activate, capability, mask;
    unsigned int shift, bits, nr;
    bool bad;
};

struct test_alias {
    struct vm_struct *area;
    pte_t pte;
};

/* EFI runtime map later drops ordinary RAM: retain a bounded early
snapshot. */
static struct {
    u64 start, end;
} crypto_ranges[128] __ro_after_init;
static unsigned int nr_crypto_ranges __ro_after_init;
```

何をしているか

使用するAPI、MSR番号、CPU設定とaliasの構造体、EFI領域の保存配列を定義する。

なぜ必要か

鍵の位置・利用条件・一時mapping・対象RAM領域を実験専用の状態として保持するため。

mktme_test_save_efi_map

```
void __init mktme_test_save_efi_map(void)
{
    efi_memory_desc_t *md;
```

```
nr_crypto_ranges = 0;
if (!efi_enabled(EFI_MEMMAP))
    return;
for_each_efi_memory_desc(md) {
    u64 bytes;

    if (!(md->attribute & TEST_EFI_CPU_CRYPT0) ||
        !(md->attribute & EFI_MEMORY_WB))
        continue;
    if (nr_crypto_ranges == ARRAY_SIZE(crypto_ranges) ||
        md->num_pages > (U64_MAX >> EFI_PAGE_SHIFT))
        goto invalid;
    bytes = md->num_pages << EFI_PAGE_SHIFT;
    if (md->phys_addr > U64_MAX - bytes)
        goto invalid;
    crypto_ranges[nr_crypto_ranges].start = md->phys_addr;
    crypto_ranges[nr_crypto_ranges++].end = md->phys_addr +
bytes;
    }
    return;
invalid:
    nr_crypto_ranges = 0;
    pr_err("EFI encryption range snapshot overflow; experiment
disabled\n");
}
```

何をしているか

暗号化可能かつWB属性のEFI領域を最大128個保存する。範囲数・サイズ・終端のoverflow時は保存数をゼロにする。

なぜ必要か

確保した実験ページ全体が暗号化可能なRAMに含まれるかをopen時に確認するため。

test_check_cpu

```
/* Called on every online CPU under the CPU hotplug read lock. */
```

```
static void test_check_cpu(void *arg)
{
    struct test_config *cfg = arg;
    struct cpuinfo_x86 *c = &cpu_data(smp_processor_id());
    u64 act, cap, status;

    /* RFC #51: only these models' alias prohibition MSR is audited
    here. */
    if (c->x86_vendor != X86_VENDOR_INTEL || c->x86 != 6 ||
        (c->x86_model != 0x6a && c->x86_model != 0x6c) ||
        !cpu_has(c, X86_FEATURE_TME) ||
        !cpu_has(c, X86_FEATURE_PCONFIG) ||
        !cpu_has(c, X86_FEATURE_CLFLUSH) ||
        cpu_has(c, X86_FEATURE_HYPERVISOR) ||
        rdmsrl_safe(TEST_TME_ACTIVATE, &act) ||
        rdmsrl_safe(TEST_TME_CAPABILITY, &cap) ||
        rdmsrl_safe(TEST_ICX_STATUS, &status) ||
        (status & BIT_ULL(8)) || act != cfg->activate ||
        cap != cfg->capability || c->x86_phys_bits != cfg->shift)
        WRITE_ONCE(cfg->bad, true);
}
```

何をしているか

CPUモデル、TME・PCONFIG・CLFLUSH、MSR値、alias禁止状態、物理アドレス幅を確認し、不一致をcfg->badに記録する。

なぜ必要か

KeyID aliasアクセスの前提を満たさないCPUで実験処理を実行しないため。

test_configure

```
static int test_configure(struct test_config *cfg)
{
    unsigned int width;

    if (!intel_mktme_test_enabled() ||
```

```

    !boot_cpu_has(X86_FEATURE_TME) ||
    !boot_cpu_has(X86_FEATURE_PCONFIG) ||
    !pconfig_target_supported(MKTME_TARGET) ||
    topology_max_packages() != 1 || num_node_state(N_MEMORY) !=
1)

    return -EOPNOTSUPP;
    if (rdmsrl_safe(TEST_TME_ACTIVATE, &cfg->activate) ||
        rdmsrl_safe(TEST_TME_CAPABILITY, &cfg->capability))
        return -EIO;
    cfg->bits = (cfg->activate >> 32) & 15;
    width = cpuid_eax(0x80000008) & 255;
    /* Locked, enabled, Key0 TME active, AES-XTS-128 activated/
supported. */
    if ((cfg->activate & 3) != 3 || (cfg->activate & BIT_ULL(31)) ||
        ((cfg->activate >> 4) & 15) ||
        !(cfg->activate & BIT_ULL(48)) || !(cfg->capability & 1) ||
        cfg->bits < 2 || cfg->bits > ((cfg->capability >> 32) & 15)
    ||
        width > __PHYSICAL_MASK_SHIFT || width <= cfg->bits +
PAGE_SHIFT)
        return -EOPNOTSUPP;
    cfg->shift = width - cfg->bits;
    cfg->mask = GENMASK_ULL(width - 1, cfg->shift);
    cfg->nr = min_t(unsigned int, (1U << cfg->bits) - 1,
        (cfg->capability >> 36) & 0x7fff);
    if (cfg->nr < 2 || (physical_mask & cfg->mask) ||
        (physical_mask & GENMASK_ULL(cfg->shift - 1, 0)) !=
        GENMASK_ULL(cfg->shift - 1, 0))
        return -EINVAL;
    on_each_cpu(test_check_cpu, cfg, 1);
    if (cfg->bad)
        return -EOPNOTSUPP;
    return 0;
}

```

何をしているか

MKTME状態、単一package・memory node、MSRの有効化・algorithm・KeyID数を確認する。

KeyIDのshift/maskを計算し、physical_maskと全online CPUの状態を照合する。

なぜ必要か

KeyID 1/2が利用でき、Stage 1のmaskと実際のCPU設定が一致する環境だけで実験を開始するため。

mktme_test_make_pte

```
static int mktme_test_make_pte(const struct test_config *cfg,
                               unsigned long pfn, unsigned int keyid, pte_t *out)
{
    u64 pa;
    pte_t base;

    if (!keyid || keyid > cfg->nr || pfn > (physical_mask >>
PAGE_SHIFT))
        return -EINVAL;
    pa = (u64)pfn << PAGE_SHIFT;
    if (pa & ~GENMASK_ULL(cfg->shift - 1, PAGE_SHIFT))
        return -EINVAL;
    base = pfn_pte(pfn, PAGE_KERNEL);
    /* KeyID belongs to the PTE address field, never to normal PFN/
prot. */
    *out = __pte(pte_val(base) | ((u64)keyid << cfg->shift));
    if (pte_pfn(*out) != pfn ||
        (pte_val(*out) & cfg->mask) != (u64)keyid << cfg->shift)
        return -EINVAL;
    return 0;
}
```

何をしているか

PFNとKeyIDの範囲を確認し、pfn_pte()で作ったPTEのアドレス領域にKeyIDを加える。pte_pfn()が元のPFNを返すことも確認する。

なぜ必要か

通常PFNにKeyIDを混ぜず、同じ物理ページを異なるKeyIDで参照するPTEを作るため。

test_install_pte

```
static int test_install_pte(pte_t *ptep, pgtable_t token,
                           unsigned long addr, void *arg)
{
    struct test_alias *alias = arg;

    if (!pte_none(*ptep))
        return -EBUSY;
    set_pte_at(&init_mm, addr, ptep, alias->pte);
    return 0;
}
```

何をしているか

空のPTEだけを対象に、set_pte_at()で用意したPTEをinit_mmへ設定する。

なぜ必要か

一時alias用の仮想アドレスにKeyID付きPTEを設置し、既存mappingの上書きを避けるため。

mktme_test_map_pfn

```
/* Private helper: caller owns/pins this PFN until all aliases are
removed. */
static int mktme_test_map_pfn(const struct test_config *cfg,
                              unsigned long pfn, unsigned int keyid, struct test_alias
*alias)
{
    int ret;

    ret = mktme_test_make_pte(cfg, pfn, keyid, &alias->pte);
    if (ret)
        return ret;
    alias->area = get_vm_area(PAGE_SIZE, VM_IOREMAP);
    if (!alias->area)
```

```
        return -ENOMEM;
    ret = apply_to_page_range(&init_mm, (unsigned long)alias->area->addr,
                             PAGE_SIZE, test_install_pte, alias);
    if (ret) {
        unmap_kernel_range((unsigned long)alias->area->addr,
                           PAGE_SIZE);
        free_vm_area(alias->area);
        alias->area = NULL;
        return ret;
    }
    flush_cache_vmap((unsigned long)alias->area->addr,
                     (unsigned long)alias->area->addr + PAGE_SIZE);
    return 0;
}
```

何をしているか

1ページの仮想領域を予約し、KeyID付きPTEを設置する。失敗時はmappingと予約領域を解除し、成功時はflush_cache_vmap()を呼ぶ。

なぜ必要か

専用ページへの一時aliasを作り、通常のdirect mapやページ所有情報を変更せずにアクセスするため。

mktme_test_unmap

```
static void mktme_test_unmap(struct test_alias *alias)
{
    if (!alias->area)
        return;
    /* RFC #14: flush through the old KeyID before removing its PTE.
     */
    clflush_cache_range(alias->area->addr, PAGE_SIZE);
    unmap_kernel_range((unsigned long)alias->area->addr, PAGE_SIZE);
    free_vm_area(alias->area);
    alias->area = NULL;
}
```

```
}
```

何をしているか

旧KeyIDのmappingが有効な間に全ページをclflushし、unmap_kernel_range()でPTEとTLBを処理して仮想領域を解放する。

なぜ必要か

異なるKeyIDへの切替前やページ解放前に、以前のaliasのcacheとmappingを残さないため。

program_test_key

```
static int program_test_key(struct mktme_key_program *prog,
    unsigned int keyid, bool clear)
{
    int status;

    if (keyid != 1 && keyid != 2)
        return -EINVAL;
    memset(prog, 0, sizeof(*prog));
    prog->keyid = keyid;
    /* RFC #23: even CLEAR_KEY needs an activated algorithm. */
    prog->keyid_ctrl = MKTME_AES_XTS_128 |
        (clear ? MKTME_KEYID_CLEAR_KEY : MKTME_KEYID_SET_KEY_RANDOM);
    status = mktme_key_program(prog);
    if (status != MKTME_PROG_SUCCESS)
        pr_err("Key%u %s PCONFIG status=%d\n", keyid,
            clear ? "clear" : "program", status);
    /* Bounded experiment: no hidden retry or partial-success
    acceptance. */
    return status == MKTME_PROG_SUCCESS ? 0 : -EIO;
}
```

何をしているか

KeyIDを1/2に限定し、AES-XTS-128とrandom key設定または鍵消去を指定してmktme_key_program()を呼ぶ。成功以外は-EIOを返す。

なぜ必要か

PTEのKeyID設定に加えてPCONFIGで実際の鍵設定・消去を行うため。

test_page_encryptable

```
static bool test_page_encryptable(struct page *page)
{
    u64 pa = page_to_phys(page);
    unsigned int i;

    for (i = 0; i < nr_crypto_ranges; i++) {
        if (pa >= crypto_ranges[i].start &&
            pa < crypto_ranges[i].end &&
            crypto_ranges[i].end - pa >= PAGE_SIZE)
            return true;
    }
    return false;
}
```

何をしているか

ページの物理アドレスから、1ページ全体が保存済みEFI領域に収まるかを調べる。

なぜ必要か

暗号化可能と記録されたRAMだけを実験対象にするため。

test_session • test_lock • 状態変数

```
/* One session owns KeyIDs 1/2 and one payload page until FINISH/last
close. */
struct test_session {
    struct test_config cfg;
    struct page *page;
    struct mktme_key_program *prog;
    struct mktme_test_info info;
```

```
    unsigned int attempted;
    unsigned int read_key;
};

static DEFINE_MUTEX(test_lock);
static bool active, poisoned, sleeping;

/* Caller holds test_lock. Pin the worker while CPU hotplug is
excluded. */
```

何をしているか

専用ページ、鍵operand、設定、取得情報、読出し鍵をsessionにまとめる。mutexとactive/poisoned/sleepingで共有状態を管理する。

なぜ必要か

同時操作を直列化し、ページと鍵をsession終了まで保持するため。

test_on_cpu

```
static long test_on_cpu(long (*fn)(void *), void *arg)
{
    long ret;

    get_online_cpus();
    ret = work_on_cpu(cpumask_first(cpu_online_mask), fn, arg);
    put_online_cpus();
    return ret;
}
```

何をしているか

CPU hotplugのread lockを保持し、最初のonline CPUで指定関数を同期実行する。

なぜ必要か

鍵操作とページアクセスの実行CPUを処理中に固定するため。

test_close_session

```
static long test_close_session(void *arg)
{
    struct test_session *s = arg;
    unsigned int keyid;
    int ret = 0;

    /* No temporary alias survives a read/write, including their
errors. */
    if (s->page) {
        clear_page(page_address(s->page));
        clflush_cache_range(page_address(s->page), PAGE_SIZE);
    }
    if (s->attempted) {
        test_check_cpu(&s->cfg);
        if (s->cfg.bad) {
            ret = -EIO;
        } else {
            for (keyid = 1; keyid <= s->attempted; keyid++)
                if (program_test_key(s->prog, keyid, true))
                    ret = -EIO;
        }
    }
    if (ret) {
        poisoned = true;
        pr_err("key cleanup failed; device disabled until reboot\n");
    }
    if (s->page)
        __free_page(s->page);
    if (s->prog) {
        memzero_explicit(s->prog, PAGE_SIZE);
        free_page((unsigned long)s->prog);
    }
    s->page = NULL;
    s->prog = NULL;
    s->attempted = 0;
    return ret;
}
```

```
}
```

何をしているか

alias解除済みのページをKey0経由でゼロ化・flushし、設定を試みた鍵を消去する。失敗時はデバイスを使用不可にし、ページとPCONFIG用領域を解放する。

なぜ必要か

FINISH、最終close、open途中の失敗で専用ページと鍵の後片付けを共通化するため。

test_open_session

```
static long test_open_session(void *arg)
{
    struct test_session *s = arg;
    unsigned int keyid;
    int ret;

    ret = test_configure(&s->cfg);
    if (ret)
        return ret;
    /* Page allocation also satisfies PCONFIG's 256-byte alignment.
    */
    BUILD_BUG_ON(sizeof(*s->prog) > PAGE_SIZE);
    BUILD_BUG_ON(offsetof(struct mktme_key_program, keyid_ctrl) !=
2);
    BUILD_BUG_ON(offsetof(struct mktme_key_program, key_field_1) !=
64);
    BUILD_BUG_ON(MKTME_TEST_MAX_BYTES != PAGE_SIZE);
    s->prog = (void *)get_zeroed_page(GFP_KERNEL);
    if (!s->prog)
        return -ENOMEM;
    s->page = alloc_page(GFP_KERNEL | __GFP_ZERO);
    if (!s->page) {
        ret = -ENOMEM;
        goto fail;
    }
}
```

```
if (!test_page_encryptable(s->page)) {
    ret = -EOPNOTSUPP;
    goto fail;
}
for (keyid = 1; keyid <= 2; keyid++) {
    s->attempted = keyid;
    ret = program_test_key(s->prog, keyid, false);
    if (ret)
        goto fail;
}
clflush_cache_range(page_address(s->page), PAGE_SIZE);
s->info.pfn = page_to_pfn(s->page);
s->read_key = 1;
return 0;
fail:
    test_close_session(s);
    return ret;
}
```

何をしているか

環境を確認し、PCONFIG用領域と専用ページを確保する。ページの暗号化可否を確認してKey1/2を設定し、Key0のcacheをflushしてPFNを保存する。失敗時は後片付けする。

なぜ必要か

1つのsessionで同じページと2つの鍵を保持して実験するため。

test_io

```
struct test_io {
    struct test_session *session;
    void *buffer;
    size_t length;
    bool writing;
};
```

何をしているか

session、kernel buffer、長さ、転送方向をworkerへ渡す構造体を定義する。

なぜ必要か

ユーザーコピーとKeyID mapping経由のアクセスを分けるため。

test_access

```
static long test_access(void *arg)
{
    struct test_io *io = arg;
    struct test_session *s = io->session;
    struct test_alias alias = { };
    unsigned int keyid = io->writing ? 1 : s->read_key;
    int ret;

    /* Recheck the execution CPU after any hotplug between syscalls.
    */
    test_check_cpu(&s->cfg);
    if (s->cfg.bad || !intel_mktme_test_enabled())
        return -EOPNOTSUPP;
    ret = mktme_test_map_pfn(&s->cfg, page_to_pfn(s->page), keyid,
&alias);
    if (ret)
        return ret;
    if (io->writing) {
        memset(alias.area->addr, 0, PAGE_SIZE);
        memcpy(alias.area->addr, io->buffer, io->length);
        s->info.length = io->length;
    } else {
        memcpy(io->buffer, alias.area->addr, io->length);
    }
    s->info.keyid = keyid;
    s->info.pte = pte_val(alias.pte);
    mktme_test_unmap(&alias);
    return 0;
}
```

何をしているか

CPU状態を再確認し、sessionの同じpage_to_pfn()をwrite時はKey1、read時は選択した鍵でmapする。データをコピーし、使用PTEとKeyIDを保存してunmapする。

なぜ必要か

ページを交換せずKey1 write・Key1 read・Key2 readを実行し、操作ごとにcache/TLBの後片付けを完了するため。

test_open

```
static int test_open(struct inode *inode, struct file *file)
{
    struct test_session *s;
    int ret;

    if (!capable(CAP_SYS_RAWIO))
        return -EPERM;
    if (mutex_lock_interruptible(&test_lock))
        return -ERESTARTSYS;
    ret = -EBUSY;
    if (active || sleeping)
        goto out;
    ret = -EIO;
    if (poisoned)
        goto out;
    ret = -ENOMEM;
    s = kzalloc(sizeof(*s), GFP_KERNEL);
    if (!s)
        goto out;
    ret = test_on_cpu(test_open_session, s);
    if (ret) {
        kfree(s);
        goto out;
    }
    file->private_data = s;
    active = true;
```

```
    ret = nonseekable_open(inode, file);
out:
    mutex_unlock(&test_lock);
    return ret;
}
```

何をしているか

CAP_SYS_RAWIOを確認し、同時open・休止移行中・鍵cleanup失敗後の利用を拒否する。
sessionを作り、初期化成功後にfile->private_dataへ保持する。

なぜ必要か

共有KeyID 1/2を1つの実験sessionだけで使用するため。

test_release

```
static int test_release(struct inode *inode, struct file *file)
{
    struct test_session *s = file->private_data;
    int ret = 0;

    mutex_lock(&test_lock);
    if (s->prog)
        ret = test_on_cpu(test_close_session, s);
    kfree(s);
    active = false;
    mutex_unlock(&test_lock);
    return ret;
}
```

何をしているか

mutexの下で必要なsession cleanupを行い、sessionを解放してactiveを解除する。

なぜ必要か

FINISHを呼ばずに最後のファイル参照を閉じた場合も、専用ページと鍵を後片付けするため。

test_transfer

```
/* Record semantics: each write replaces data; each read starts at
byte zero. */
static ssize_t test_transfer(struct file *file, void __user *user,
                             size_t count, bool writing)
{
    struct test_session *s = file->private_data;
    struct test_io io = { .session = s, .writing = writing };
    ssize_t ret;

    if (count > MKTME_TEST_MAX_BYTES)
        return -EMSGSIZE;
    if (!count)
        return 0;
    if (mutex_lock_interruptible(&test_lock))
        return -ERESTARTSYS;
    ret = -ENODEV;
    if (!s->page)
        goto out;
    ret = -ENODATA;
    if (!writing && !s->info.length)
        goto out;
    io.length = writing ? count : min_t(size_t, count, s-
>info.length);
    ret = -ENOMEM;
    io.buffer = kmalloc(io.length, GFP_KERNEL);
    if (!io.buffer)
        goto out;
    ret = -EFAULT;
    /* User copies must run in the caller's mm, never in
work_on_cpu(). */
    if (writing && copy_from_user(io.buffer, user, io.length))
        goto free_buffer;
    ret = test_on_cpu(test_access, &io);
    if (ret)
        goto free_buffer;
    if (!writing && copy_to_user(user, io.buffer, io.length)) {
```

```
        ret = -EFAULT;
        goto free_buffer;
    }
    ret = io.length;
free_buffer:
    kfree(io.buffer);
out:
    mutex_unlock(&test_lock);
    return ret;
}
```

何をしているか

長さやsession状態を確認し、一時bufferを介してcopy_from_user()/copy_to_user()を行う。実ページへの転送はtest_access()に任せ、最後にbufferを解放する。

なぜ必要か

ユーザーアドレスをworkerで直接扱わず、上限付きのバイナリデータをKeyID mappingへ受け渡すため。

test_read

```
static ssize_t test_read(struct file *file, char __user *buf,
                        size_t count, loff_t *pos)
{
    return test_transfer(file, buf, count, false);
}
```

何をしているか

test_transfer()を読出しモードで呼ぶ。

なぜ必要か

read()を選択済みKeyIDによる同じページの読出しに接続するため。

test_write

```
static ssize_t test_write(struct file *file, const char __user *buf,
                          size_t count, loff_t *pos)
{
    return test_transfer(file, (void __user *)buf, count, true);
}
```

何をしているか

test_transfer()を書込みモードで呼ぶ。

なぜ必要か

write()をKey1による専用ページへの書込みに接続するため。

test_ioctl

```
static long test_ioctl(struct file *file, unsigned int cmd, unsigned
long arg)
{
    struct test_session *s = file->private_data;
    void __user *user = (void __user *)arg;
    __u32 keyid;
    long ret = 0;

    if (mutex_lock_interruptible(&test_lock))
        return -ERESTARTSYS;
    if (!s->page) {
        ret = -ENODEV;
        goto out;
    }
    switch (cmd) {
    case MKTME_TEST_SET_READ_KEY:
        if (copy_from_user(&keyid, user, sizeof(keyid)))
            ret = -EFAULT;
        else if (keyid != 1 && keyid != 2)
            ret = -EINVAL;
    }
```

```
        else
            s->read_key = keyid;
        break;
    case MKTME_TEST_GET_INFO:
        if (copy_to_user(user, &s->info, sizeof(s->info)))
            ret = -EFAULT;
        break;
    case MKTME_TEST_FINISH:
        ret = test_on_cpu(test_close_session, s);
        break;
    default:
        ret = -ENOTTY;
    }
out:
    mutex_unlock(&test_lock);
    return ret;
}
```

何をしているか

読出しKeyIDを1/2から選択し、PFN・最後のPTE等をuserspaceへ返し、FINISHでsessionを終了する。不明な操作や不正なKeyIDを拒否する。

なぜ必要か

同じページへの読出し鍵の切替、使用mappingの確認、明示的な後片付けを提供するため。

test_compat_ioctl

```
#ifdef CONFIG_COMPAT
static long test_compat_ioctl(struct file *file, unsigned int cmd,
                             unsigned long arg)
{
    return test_ioctl(file, cmd, (unsigned long)compat_ptr(arg));
}
#endif
```

何をしているか

compat_ptr()でユーザーポインタを変換し、共通のtest_ioctl()へ渡す。

なぜ必要か

32-bit userspaceからも同じ固定幅の実験interfaceを使用するため。

test_fops • test_device

```
static const struct file_operations test_fops = {
    .owner = THIS_MODULE,
    .open = test_open,
    .release = test_release,
    .read = test_read,
    .write = test_write,
    .unlocked_ioctl = test_ioctl,
#ifdef CONFIG_COMPAT
    .compat_ioctl = test_compat_ioctl,
#endif
    .llseek = no_llseek,
};

static struct miscdevice test_device = {
    .minor = MISC_DYNAMIC_MINOR,
    .name = "mktme_test",
    .fops = &test_fops,
    .mode = 0600,
};

/* A session must not survive a suspend/hibernate that can replace
its keys. */
```

何をしているか

open/release/read/write/ioctlを実装関数に結び付け、名前mktme_test、mode 0600のmiscdeviceを定義する。

なぜ必要か

userspaceのファイル操作を専用ページの実験処理へ接続するため。

test_pm_event

```
static int test_pm_event(struct notifier_block *nb, unsigned long
event,
                        void *unused)
{
    int ret = NOTIFY_OK;

    mutex_lock(&test_lock);
    switch (event) {
    case PM_SUSPEND_PREPARE:
    case PM_HIBERNATION_PREPARE:
    case PM_RESTORE_PREPARE:
        if (active)
            ret = NOTIFY_BAD;
        else
            sleeping = true;
        break;
    case PM_POST_SUSPEND:
    case PM_POST_HIBERNATION:
    case PM_POST_RESTORE:
        sleeping = false;
        break;
    }
    mutex_unlock(&test_lock);
    return ret;
}
```

何をしているか

sessionがactiveならsuspend等への移行を拒否し、移行中はsleeping状態を設定する。復帰通知で解除する。

なぜ必要か

鍵が置き換わり得る電源状態の変化を、実験sessionや新規openと重ねないため。

test_pm_notifier

```
static struct notifier_block test_pm_notifier = {  
    .notifier_call = test_pm_event,  
};
```

何をしているか

電源管理通知のコールバックをtest_pm_eventに設定する。

なぜ必要か

実験中の電源状態移行をデバイスの状態と連動させるため。

mktme_test_init

```
static int __init mktme_test_init(void)  
{  
    int ret;  
  
    ret = register_pm_notifier(&test_pm_notifier);  
    if (ret)  
        return ret;  
    ret = misc_register(&test_device);  
    if (ret)  
        unregister_pm_notifier(&test_pm_notifier);  
    return ret;  
}
```

何をしているか

電源管理notifierを登録した後、misc_register()でデバイスを登録する。デバイス登録失敗時はnotifierを解除する。

なぜ必要か

/dev/mktme_testから各file operationを呼び出せるようにするため。

late_initcall(mktme_test_init)

```
late_initcall(mktme_test_init);
```

何をしているか

mktme_test_initをlate initの呼出し対象にする。

なぜ必要か

組み込みの実験デバイスを登録するため。

tools/mktme_test/Makefile

変更コード

```
# SPDX-License-Identifier: GPL-2.0
CC ?= cc
CFLAGS ?= -O2 -g
CFLAGS += -Wall -Wextra -Werror -std=gnu11

.PHONY: all clean
all: mktme_test_user

mktme_test_user: mktme_test_user.c ../../include/uapi/linux/
mktme_test.h
    $(CC) $(CPPFLAGS) $(CFLAGS) $(LDFLAGS) -o $@ $<

clean:
    $(RM) mktme_test_user
```

何をしているか

userspace Cプログラムと共有headerを依存対象とし、コンパイラ・警告option・出力名・clean対象を指定する。

なぜ必要か

実験デバイスの共有interfaceを使うuserspaceプログラムを単独の実行ファイルにするため。

tools/mktme_test/mktme_test_user.c

変更コード

```
// SPDX-License-Identifier: GPL-2.0
#include <errno.h>
#include <fcntl.h>
#include <inttypes.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/ioctl.h>
#include <unistd.h>
#include "../include/uapi/linux/mktme_test.h"

static void dump(const char *label, const unsigned char *data, size_t
length)
{
    size_t i;

    printf("%s:\n", label);
    for (i = 0; i < length; i++)
        printf("%02x%c", data[i], (i + 1) % 16 && i + 1 < length ? '
' : '\n');
}

static int command(int fd, unsigned long op, void *arg)
{
    int ret;

    do {
        ret = ioctl(fd, op, arg);
    } while (ret < 0 && errno == EINTR);
    return ret;
}
```

```
static int get_info(int fd, unsigned int keyid, uint64_t *pfn, int
first)
{
    struct mktme_test_info info;

    if (command(fd, MKTME_TEST_GET_INFO, &info) < 0) {
        perror("GET_INFO");
        return -1;
    }
    if (info.keyid != keyid || (!first && info.pfn != *pfn)) {
        fprintf(stderr, "Unexpected PFN/KeyID in device
information\n");
        return -1;
    }
    *pfn = info.pfn;
    printf("PFN=0x%" PRIx64 " KeyID=%u installed PTE=0x%" PRIx64
"\n",
           (uint64_t)info.pfn, info.keyid, (uint64_t)info.pte);
    return 0;
}

static int read_key(int fd, __u32 keyid, unsigned char *data, size_t
length,
                    uint64_t *pfn)
{
    ssize_t n;

    if (command(fd, MKTME_TEST_SET_READ_KEY, &keyid) < 0) {
        perror("SET_READ_KEY");
        return -1;
    }
    do {
        n = read(fd, data, length);
    } while (n < 0 && errno == EINTR);
    if (n < 0) {
        perror("read");
        return -1;
    }
}
```

```
    }
    /* Device reads are records from offset zero, not a byte stream.
    */
    if ((size_t)n != length) {
        fprintf(stderr, "Short read: %zd of %zu bytes\n", n, length);
        return -1;
    }
    return get_info(fd, keyid, pfn, 0);
}

int main(int argc, char **argv)
{
    const unsigned char *input;
    unsigned char key1[MKTME_TEST_MAX_BYTES],
key2[MKTME_TEST_MAX_BYTES];
    size_t length;
    uint64_t pfn = 0;
    ssize_t n;
    int fd, cmp1, cmp2, result = EXIT_FAILURE;

    if (argc > 2) {
        fprintf(stderr, "Usage: %s [text, default: Hello]\n",
argv[0]);
        return EXIT_FAILURE;
    }
    input = (const unsigned char *)(argc == 2 ? argv[1] : "Hello");
    length = strlen((const char *)input);
    if (!length || length > MKTME_TEST_MAX_BYTES) {
        fprintf(stderr, "Input must contain 1..%u bytes\n",
MKTME_TEST_MAX_BYTES);
        return EXIT_FAILURE;
    }

    /* 1. open allocates one page and programs both keys. */
    fd = open("/dev/mktme_test", O_RDWR | O_CLOEXEC);
    if (fd < 0) {
        perror("open /dev/mktme_test");
    }
}
```

```
        return EXIT_FAILURE;
    }
    dump("Input", input, length);
    /* 2/3. write() explicitly means Key1; no write-key state is
needed. */
    do {
        n = write(fd, input, length);
    } while (n < 0 && errno == EINTR);
    if (n < 0) {
        perror("write Key1");
        goto finish;
    }
    if ((size_t)n != length) {
        fprintf(stderr, "Short write: %zd of %zu bytes\n", n,
length);
        goto finish;
    }
    if (get_info(fd, 1, &pfn, 1))
        goto finish;
    /* 4. Each device operation flushes/unmaps before returning. */
    /* 5/6. Read the same allocation with Key1. */
    if (read_key(fd, 1, key1, length, &pfn))
        goto finish;
    cmp1 = memcmp(key1, input, length);
    dump("Key1 -> Key1", key1, length);
    puts(cmp1 == 0 ? "MATCH" : "MISMATCH");
    /* 7/8. Select Key2 without allocating another page or writing
again. */
    if (read_key(fd, 2, key2, length, &pfn))
        goto finish;
    cmp2 = memcmp(key2, input, length);
    dump("Key1 -> Key2", key2, length);
    puts(cmp2 == 0 ? "MATCH" : "MISMATCH");
    result = cmp1 == 0 && cmp2 != 0 ? EXIT_SUCCESS : EXIT_FAILURE;
finish:
    /* 9. FINISH reports cleanup failure; close is also a cleanup
fallback. */
```

```
    if (command(fd, MKTME_TEST_FINISH, NULL) < 0) {
        perror("FINISH");
        result = EXIT_FAILURE;
    }
    if (close(fd) < 0) {
        perror("close");
        result = EXIT_FAILURE;
    }
    return result;
}
```

何をしているか

open()後にwrite()でKey1へ書き、ioctl()で読出し鍵を切り替えてread()する。GET_INFOで同一PFNとKeyIDを照合し、memcmp()とhex表示で各結果を比較する。FINISHとclose()で終了する。

なぜ必要か

userspaceから同じページへのKey1/Key2アクセスを順に操作し、バイナリ結果を文字列として扱わず比較するため。